Índice

Explicación técnica del proyecto **1**
1. Qué es el sistema . . . 1
2. Piezas y responsabilidades . . . 1
3. Regla central: la IA trabaja al publicar, el código trabaja al leer . . . 2
4. Modelo de datos y de certeza . . . 3
    4.1 Dos ejes independientes (VERIFICACION-Y-FUENTES.md §1) . . . 3
    4.2 Reglas de certeza . . . 3
    4.3 Procedencia (qué es humano y qué es IA) . . . 3
    4.4 Qué puede y qué no puede hacer la IA en el portal . . . 3
5. Flujo A: publicar una noticia (portal) . . . 4
6. Flujo B: el feed (código, costo 0) . . . 4
    6.1 Filtro . . . 4
    6.2 Esenciales (“Lo que debes saber”) . . . 4
    6.3 Puntaje . . . 4
    6.4 Anti-burbuja . . . 4
    6.5 Razones “¿Por qué veo esto?” . . . 5
    6.6 Aprendizaje de intereses . . . 5
    6.7 El comparador del portal . . . 5
7. Flujo C: el chat (RAG) . . . 5
8. Gateway de IA y costos . . . 5
9. Autenticación y seguridad . . . 6
10. Evidencia de calidad y de Responsible AI . . . 6
11. Stack y decisiones clave (ADRs) . . . 7
12. Stack técnico resumido . . . 8
13. Limitaciones declaradas . . . 8
14. Glosario . . . 8

Explicación técnica del proyecto

Written for: la persona del equipo que debe entender el sistema completo antes de presentarlo, sin haber escrito el código.

Documento de estudio. Cada afirmación apunta al archivo donde se puede verificar. Para lo que se muestra en vivo y en qué orden, ver GUION-PRESENTACION.md.

---

1. Qué es el sistema

App móvil (iPhone y Android, vía Expo Go) de noticias personalizadas por **ubicación simulada**. El lector elige una de 8 ubicaciones (catálogo en packages/shared/src/catalogs/locations.ts). No se usa GPS. Incluye:

- Feed ordenado por una fórmula explicable.
- Pantalla de lectura con “Cómo se hizo esta noticia”.
- Chat sobre las noticias publicadas, con fuentes y abstención.
- Portal editorial web donde una persona redacta, verifica y publica.

Restricción de presupuesto: tope de USD 20 en créditos de IA para desarrollo, pruebas y presentación (ENTREGA.md §5).

---

2. Piezas y responsabilidades

Pieza	Carpeta	Tecnología	Responsabilidad	No hace
App móvil	apps/mobile	Expo, expo-router, Firebase JS SDK	Login, ubicación, feed, lectura, señales, chat (UI)	Llamar a LLM; calcular certeza
Portal editorial	apps/admin	SvelteKit (SPA, adapter-static), Firebase Hosting	Crear, verificar, publicar; comparador; dashboard de costos	Guardar API keys de IA
API	services/api	Cloudflare Worker, Hono	Verifica identidad, gateway de IA, chat, digests, enriquecimiento, índice	Escribir en Firestore (no tiene Admin SDK, ADR-004)
Núcleo compartido	packages/shared	TypeScript, zod	Tipos, esquemas, catálogos, **motor de ranking**, reglas editoriales	Nada de UI ni de red
Base de datos	Firestore	Firebase (plan Spark)	Fuente de verdad: noticias, usuarios, eventos	Búsqueda semántica
Índice y caché	Cloudflare KV	Worker	Índice del chat (embeddings + texto recortado), digests, caché de respuestas	Ser fuente de verdad
Ledger	Cloudflare D1	Worker	Cada llamada de IA con tokens y costo; rate limit	Contenido de noticias
Embeddings	Workers AI `@cf/baai/bge-m3`	Worker	Vectores de texto multilingüe	—
LLM	Anthropic `claude-haiku-5-5`	Worker (solo vía gateway)	Enriquecer, responder chat, digest	—

Diagrama completo y tabla de responsabilidades: ARQUITECTURA.md §1–2.

Estructura del monorepo (pnpm workspaces): `apps/*`, `services/*`, `packages/*`, `evals/`, `e2e/`, `evidence/`, `docs/`.

---

3. Regla central: la IA trabaja al publicar, el código trabaja al leer

Tres puntos donde se permite IA:

1. **Publicar** (portal): sugerencias de temas, alcance, importancia, afirmaciones verificables y resumen.
2. **Chat** (app): respuesta generada con RAG, solo cuando el código decide que aporta.
3. **Digest** (resumen de la ubicación): se precalcula al publicar y se sirve desde caché.

Ninguna pantalla de lectura llama a un LLM. Razones documentadas en ADR-002:

- **Costo:** el feed se abre muchas veces por usuario; el código cuesta 0.
- **Latencia:** responde al instante y se actualiza en tiempo real.
- **Explicabilidad:** una fórmula con componentes permite decir exactamente por qué aparece cada noticia.
- **Auditoría:** el mismo motor corre en la app, en el comparador del portal y en las pruebas, con el mismo resultado.

4. Modelo de datos y de certeza

4.1 Dos ejes independientes (VERIFICACION-Y-FUENTES.md §1)

- **Workflow** (proceso interno): `borrador → en_revision → publicada | rechazada`.
- **Certeza** (lo que ve el lector): `confirmada | en_desarrollo | disputada | retractada`.

Una noticia puede estar `publicada` y `en_desarrollo` a la vez. Es lo normal en noticias de última hora.

4.2 Reglas de certeza

Están en validatePublish.ts y bloquean el botón Publicar:

Certeza	Requisito mínimo	Lo que ve el lector
`confirmada`	≥2 fuentes que confirman, de organizaciones distintas; una primaria o agencia; todas las afirmaciones respaldadas. Fuentes de redes sociales no cuentan.	Chip verde «Confirmada · N fuentes»
`en_desarrollo`	≥1 fuente que confirma y una nota de qué falta	Banner ámbar con la nota
`disputada`	≥1 fuente que confirma y ≥1 que contradice; nota que describe ambas versiones	Banner naranja, versiones lado a lado
`retractada`	Solo desde una noticia ya publicada; entrada de corrección obligatoria	Banner rojo, texto tachado, explicación; sale del feed y del chat

Checklist editorial (6 casillas, todas obligatorias): fuentes revisadas, afirmaciones con respaldo, título no sensacionalista, imagen etiquetada, alcance geográfico revisado, certeza justificada.

4.3 Procedencia (qué es humano y qué es IA)

Toda salida de IA visible lleva etiqueta de origen (VERIFICACION-Y-FUENTES.md §4):

Origen	Etiqueta visible
`original_editorial`	«Redacción: {nombre}»
`resumen_ia`	«Resumen generado con IA · revisado por {editor}» (solo si un editor lo aprobó)
`respuesta_ia`	«Respuesta generada con IA a partir de N noticias publicadas»
`portada_generada`	«Portada generada por la app (no es una fotografía)»
`ilustracion_ia`	Sello en la imagen + «No documenta el hecho» (sin proveedor, ADR-011)

4.4 Qué puede y qué no puede hacer la IA en el portal

Puede: extraer afirmaciones verificables, marcar posible sensacionalismo, sugerir temas, alcance e importancia.

No puede: marcar una afirmación como respaldada (solo el editor lo hace al vincular una fuente), asignar certeza, publicar. Las sugerencias quedan en `aiSuggestions` y el portal muestra qué aceptó y qué cambió el editor.

5. Flujo A: publicar una noticia (portal)

1. El editor crea un borrador → documento en Firestore `news/{id}` con `workflow = borrador`.
2. «Sugerir con IA» → `POST /admin/enrich` → el Worker devuelve sugerencias. Una llamada LLM por noticia. Prompt en enrich.v2.ts.
3. El editor acepta o edita las sugerencias, carga fuentes, resuelve la imagen y completa el checklist.
4. Elige certeza; `validatePublish` valida las reglas de §4.2.
5. Publica → Firestore `workflow = publicada` y `POST /admin/index/upsert`.
6. El Worker calcula el embedding (Workers AI) y actualiza el índice en KV. Invalida digests afectados.
7. Firestore empuja la noticia a todos los teléfonos con listener en tiempo real.

Validación de entrada con zod en endpoints y formularios (schemas.ts).

---

6. Flujo B: el feed (código, costo 0)

Entrada: rankFeed.ts. Salida: `mustKnow` (esenciales), `feed` (ordenado, con razones) y `diversity` (conteo de temas y alcances en el top 10).

6.1 Filtro

Descarta noticias que no estén `publicada`, que estén `retractada`, con fecha inválida o con fecha futura.

6.2 Esenciales ("Lo que debes saber")

antibubble.ts `isEssential`: importancia 3, publicada dentro de la ventana de `MUST_KNOW_WINDOW_HOURS` (72 h, config/feed.ts), y cercanía ≥ 0.4 o alcance global. Entran siempre, con puntaje **neutro** (sin personalizar), ordenadas por fecha.

6.3 Puntaje

score.ts:

`score = (wI·importancia/3 + wG·cercanía + wA·afinidad + wR·frescura) × penalización`

Los pesos se normalizan para sumar 1. Si el usuario desactiva personalización, `wA` pasa a 0.

Componente	Función	Valores
Importancia	`importance / 3`	0, 1/3, 2/3 o 1
Cercanía (proximity.ts)	Qué tan relacionada está con la ubicación	Misma ciudad 1; mismo país 0.45 (alcance local) o 0.75 (nacional); global 0.6; región 0.4; internacional 0.2; resto 0.1
Afinidad (affinity.ts)	Intereses aprendidos del usuario	Promedio de intereses por tema (0–10 normalizado). Tema silenciado = 0. Sin datos = 0.3
Frescura (recency.ts)	Decaimiento exponencial por horas	Vida media de 24 h si importancia ≥ 2; si no, 18 h
Penalización	Ya leída	×0.35

El resultado se limita a [0, 1]. Desempates: puntaje, luego fecha, luego ID (orden estable entre dispositivos).

6.4 Anti-burbuja

Dos mecanismos en antibubble.ts:

- **Cuotas:** asegura al menos una noticia nacional y una internacional en el top 10, en posiciones 5 y 8 si hace falta.
- **Diversidad:** evita tres seguidas del mismo tema principal. Reserva separadores para que no quede una cola de deportes ni una cuota sin separar. Hubo dos fallas en 182 pruebas antes de corregirlo (LOOP-009).

6.5 Razones "¿Por qué veo esto?"

explain/reasons.ts toma los componentes del puntaje y genera el texto. Como sale del mismo cálculo, la explicación es fiel por construcción. Catálogo incompleto de razones corregido en LOOP-010.

6.6 Aprendizaje de intereses

Cada interacción (abrir, tiempo de lectura, "menos de esto") se guarda en `users/{uid}/events`. Las reglas en interests/update.ts actualizan `users/{uid}.interests` de forma determinista. No hay LLM en este paso.

6.7 El comparador del portal

comparator.ts llama al mismo `rankFeed` para las 8 ubicaciones y muestra en qué posición y con qué tier cae cada noticia. Sirve para validar la personalización sin teléfonos.

---

7. Flujo C: el chat (RAG)

Endpoint `POST /chat`, en services/api/src/rag/chat.ts. Pasos en orden:

1. **Rate limit** por usuario, en D1 (`CHAT_RATE_LIMIT_PER_HOUR`). En entorno dev, los administradores quedan exentos para poder correr el set dorado.
2. **Flags y presupuesto** (`gateway.chatPolicy`): si el kill switch está activo, responde `blocked`. Si el gasto alcanzó el umbral, cambia a `retrieval_only`: muestra fuentes encontradas sin texto generado.
3. **Intención por reglas** (packages/shared/src/chat/intent.ts): tema, mi región, otro país, resumen, etc. Sin LLM.
4. **Digest:** si la intención es «resumen» o «mi región» y es genérica, devuelve el digest precalculado de la ubicación. Costo 0 desde la segunda consulta.
5. **Caché KV** (15 min): la clave es un SHA-256 de la configuración, la versión del prompt, el modo, la pregunta normalizada, la ubicación, la versión del índice y el historial. Si hay acierto, devuelve sin gastar ni buscar.
6. **Recuperación:** embedding de la pregunta (Workers AI) → búsqueda por coseno contra todo el índice en KV (search.ts, fuerza bruta). Top 6. Excluye retractadas. Para «otro país» filtra por país; para «mi región» aplica un bono de orden de 0.1 (no cambia el umbral).
7. **Abstención:** si ningún resultado llega a $\tau = 0.45$, responde «no tengo información» **sin llamar al LLM** y registra la fila `abstained` con costo 0.
8. **Generación:** una llamada a Claude Haiku 5.5 con las noticias del top, cada una recortada a 600 caracteres (chatConfig.ts). Prompt `chat.v2` en chat.v2.ts. Salida estructurada en bloques.
9. **Validación en código:** cada cita debe existir en lo recuperado. Si el modelo no devuelve bloques válidos, la respuesta es abstención. Si el proveedor falla, cae a `retrieval_only`.
10. **Certeza y fuentes** se toman de la base de datos, nunca del modelo. Así el chat hereda «en desarrollo» o «disputada» tal cual.
11. Se guarda en caché (si no fue un fallo degradado) y se responde con la etiqueta de IA.

Qué garantiza y qué no: CHAT-RAG.md. Resultados: set dorado de 34 casos y set de inyección de 3, ver §10.

---

8. Gateway de IA y costos

Archivo central: services/api/src/ai/gateway.ts. Toda llamada a un modelo pasa por `gateway.run(task, input, ctx)`.

Por cada llamada, el gateway:

1. Lee flags (KV) y gasto total (D1) en paralelo.
2. Evalúa el nivel de presupuesto. Si hay kill switch o si la tarea está bloqueada por nivel, registra `blocked_budget` y lanza error.
3. Resuelve proveedor y modelo por tarea (config.ts).
4. Verifica que el modelo tenga precio configurado (pricing.ts).
5. Aplica rate limit si es chat.
6. Llama al proveedor (`anthropic`, `workers-ai` o `mock`).
7. Calcula costo con tokens reales y registra en D1 (`ai_calls`): tarea, proveedor, modelo, tokens, costo, uid, resultado.

Si el registro en D1 falla, no tumba la respuesta: deja el registro en logs para repararlo después.

Tipos de resultado (`outcome`): `ok`, `error`, `blocked_budget`, `abstained`, y `cached` como bandera.

Frontera verificada: boundary.test.ts falla si algún archivo fuera de `src/ai` importa un SDK de proveedor.

Modo mock: `AI_MODE=mock` devuelve respuestas fijas para desarrollo sin gastar créditos.

Precios medidos (de PRESUPUESTO-IA §4 y F4-08): `chat_answer` USD 0.000296 por llamada; `digest` USD 0.000809; `enrich` USD 0.003183 (medido con Haiku 4.5; con 5.5 sin medir).

Resultado del gasto real (ENTREGA.md §5): USD 0.0605 según el ledger, USD 0.06 según la consola de Anthropic. Reserva de USD 7 para la demo, intacta. Estimación de la demo: USD 0.075.

---

9. Autenticación y seguridad

Caso	Mecanismo	Detalle
Portal	Google con `signInWithPopup`	Escribe solo si existe `admins/{uid}` en Firestore (firestore.rules)
App móvil	Puente web	Expo Go no tiene Google nativo. La app abre `/auth/mobile` en el navegador; la página devuelve el `id_token` por deep link en el fragmento # (nunca en query); la app llama `signInWithCredential`. ADR-003, LOOP-001
Worker	Verifica el ID token de Firebase con `jose` contra JWKS público	Exige `iss` y `aud` del proyecto. Rutas `/admin/*` además exigen `uid` en el secret `ADMIN_UIDS` (middleware/admin.ts)

Reglas de oro de seguridad (de CLAUDE.md §3): - Sin secretos en clientes. Las API keys de IA solo son secrets del Worker. - El repo es público: solo nombres de variables en `.env.example`. - Sin GPS. La ubicación es siempre la simulada que elige el usuario.

Prueba de inyección: noticia maliciosa de prueba (injection-news.json) que contiene instrucciones para el asistente. Se indexa solo para la prueba y se quita después. El modelo no las obedeció.

---

10. Evidencia de calidad y de Responsible AI

Qué	Resultado	Dónde
Pruebas automáticas	680 en verde (corrida del 2026-10-15); lint y typecheck sin errores	ENTREGA.md §1
CI en `main`	Verde el 2026-10-16	.github/workflows/ci.yml
Set dorado del chat	v4, 34 casos: abstención 5/5, 0 citas prohibidas, 0 fallos ante inyección	chat-20261013-0440.json
Latencias del chat (p50/p95)	Respuesta 2.8 s / 4.1 s; abstención 0.6 s; caché 0.3 s. Una corrida	mismo archivo; F4-08
Set de inyección	3 casos, 0 fallos	chat-injection-20261013-0419.json
Precisión de citas tras cambiar el prompt	85.7 % (24/28) → 96.0 % (24/25)	LOOP-016
Falsas abstenciones del digest	De 2/26 a 0/26 al subir el límite de tokens de salida a 1500	LOOP-015
Costo real vs panel	Ledger y consola coinciden en menos de 1 %	LOOP-003, LOOP-005
Sugerencia de IA vs decisión del editor	Importancia distinta en 5 de 5 noticias de la medición previa	LOOP-005
Matriz de Responsible AI	Cada pregunta del enunciado → decisión → pantalla → prueba	RAI-MATRIZ.md
Engineering loops	18 registros con antes y después	evidence/loops/

Cómo se mide el chat: evals/run-chat-evals.ts, métricas en evals/src/metrics.ts, casos en evals/chat-golden.jsonl e evals/chat-injection.jsonl.

---

11. Stack y decisiones clave (ADRs)

ADR	Decisión
001	Expo compatible con Expo Go (sin módulos nativos ajenos)
002	Ranking por código, no por LLM
003	Login móvil con puente web de Firebase
004	Backend en Cloudflare Workers, sin Admin SDK de Firebase
005	Búsqueda semántica por fuerza bruta en el Worker
006	La certeza la decide un humano
007	Portada tipográfica generada por código como opción preferida
008	Embeddings multilingües en Workers AI
009	Claude Haiku 4.5 por defecto (reemplazado por 012)
010	Fotos subidas a Cloudinary (plan gratuito, preset sin firma)
011	Ilustración con IA: endpoint listo, sin proveedor
012	Modelo en producción: Claude Haiku 5.5
013	La ventana del feed se ajusta el día antes de la presentación
014	Distribución de la demo: túnel con hotspot propio y Plan B nativo

Lista completa con contexto: DECISIONES.md.

Nota: ADR-012 dice que reemplaza al ADR-009, pero la entrega no dejó escrito por qué se pasó de 4.5 a 5.5 (ENTREGA.md §3). Si preguntan, esa es la respuesta honesta.

---

12. Stack técnico resumido

Capa	Tecnología
Móvil	Expo (React Native), expo-router, Firebase JS SDK, expo-web-browser
Portal	SvelteKit (SPA), Vite, Firebase Auth/Firestore, Vitest, Playwright (e2e)
API	Cloudflare Workers, Hono, zod, jose, D1, KV, Workers AI
Compartido	TypeScript estricto, zod, Vitest
IA	Anthropic `claude-haiku-5-5`; Workers AI `bge-m3` para embeddings
Infra	Firebase Spark (Auth, Firestore, Hosting), Cloudflare plan Free
Repo	pnpm workspaces, GitHub Actions

---

13. Limitaciones declaradas

Estas se dicen en voz alta en la presentación (ENTREGA.md §3–4):

- **Sin fecha del hecho.** La noticia solo tiene `publishedAt`. Una noticia vieja publicada hoy parece reciente (LOOP-017). Propuesta: campo `eventDate`, sin implementar.
- **Sin detección de duplicados** (LOOP-018).
- **Pruebas en teléfono incompletas.** Ninguna medición en dispositivo de latencia (F3-02 CA2, F3-09, F3-10). Chat sin probar en Android. Sin accesibilidad (F3-12). Lista de 55 pendientes en CAPTURAS-PENDIENTES.md.
- **Corpus de 53 noticias**, no las 55 pedidas. Tres editoriales retenidas por revisión humana.
- **Muestras pequeñas en evals:** una corrida por versión; el modelo no es determinista (Haiku 5.5 no acepta `temperature`). El umbral $\tau = 0.45$ no está calibrado con datos.
- **Demo sin ensayar completa:** 0 de 3 ensayos generales. Ver DEMO-RUNBOOK.md §6.
- **Sin proveedor de ilustración con IA.** Solo portada tipográfica y fotos libres.
- **`enrich.v2`** y el costo de `enrich` con Haiku 5.5 no están medidos.

---

14. Glosario

Término	Significado en el proyecto
RAG	Recuperar noticias relevantes (embeddings + coseno) y pasarlas al LLM como contexto
Embedding	Vector numérico de un texto; textos parecidos quedan cerca
Coseno	Medida de similitud entre dos vectores, de -1 a 1
τ (tau)	Umbral mínimo de similitud; por debajo, abstención
Abstención	Responder «no sé» sin llamar al LLM
Digest	Resumen precalculado de una ubicación, servido desde caché

Término	Significado en el proyecto
Gateway	Único punto por donde sale cualquier llamada a un modelo
Kill switch	Bandera que apaga la IA de inmediato
Retrieval only	Modo degradado: muestra fuentes sin texto generado
Esencial	Noticia importancia 3, reciente y cercana; siempre visible
Cuota	Garantía de al menos una noticia nacional e internacional en el top 10
Tier	Nivel de presentación de la tarjeta en el feed
Certeza	Lo que ve el lector sobre la confiabilidad; la pone un humano
Workflow	Estado interno del proceso editorial
Retractada	Noticia corregida públicamente; visible con corrección, fuera del feed y del chat
Puente web	Login de Google que pasa por el navegador para funcionar en Expo Go
Loop	Registro de una observación que cambió una decisión, con antes y después
Set dorado	Preguntas con respuesta esperada para medir el chat